

Modern Bootloader Systems: Working Principles, Types, and Multi-boot Capabilities

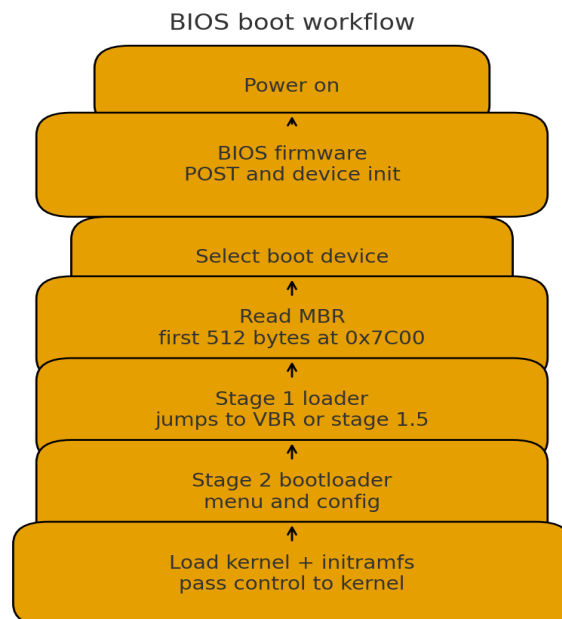
Prepared for: Operating Systems course assignment

1. Boot Workflow and Components

1.1 BIOS-based bootloaders

- 1 Power on. CPU starts executing BIOS code in ROM.
- 2 POST: memory test and basic device setup.
- 3 Boot device selection: checks the boot order.
- 4 Reads the Master Boot Record from the first disk sector into memory at address 0x7C00.
- 5 Stage 1 loader in MBR transfers to Volume Boot Record or a small stage in free sectors.
- 6 Stage 2 bootloader reads its config, shows a menu, and can load the OS kernel or chainload another loader.
- 7 OS handoff: kernel and initramfs loaded, CPU jumps to kernel entry.

Roles: BIOS initializes hardware and picks a device. MBR stores the partition table and a tiny Stage 1 program. The boot manager provides a menu, locates the kernel, and hands off control.

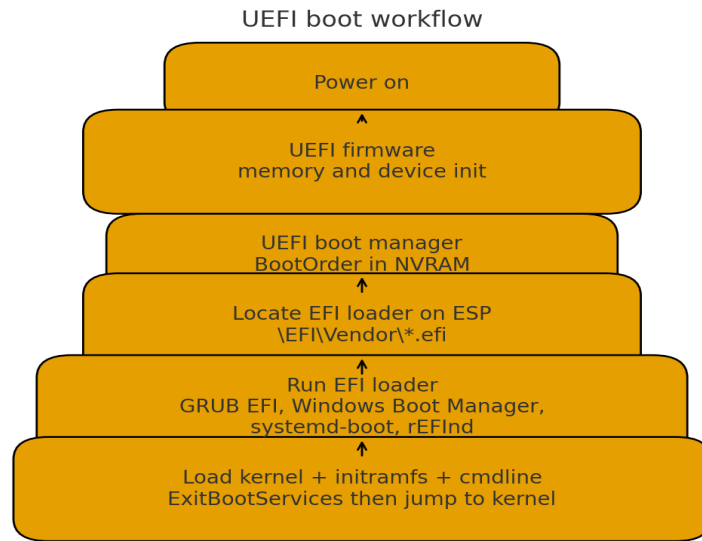


1.2 UEFI-based bootloaders

- 1 Power on. CPU runs UEFI firmware in flash memory.
- 2 Hardware and memory are initialized. UEFI drivers can access disks, network, and graphics.
- 3 UEFI boot manager reads NVRAM BootOrder and entries.
- 4 Firmware locates an EFI program on the EFI System Partition. The path looks like \EFI\Vendor\bootx64.efi.
- 5 The EFI loader runs as a normal application. Examples: Windows Boot Manager, GRUB EFI, systemd-boot, rEFInd.

- 6 The loader places kernel and initramfs in memory, sets the command line, calls ExitBootServices, and jumps to the kernel.

Roles: UEFI offers drivers, Secure Boot, and a boot manager. The EFI System Partition stores .efi loaders. The boot manager selects and runs the loader that finally hands off to the kernel.



2. Architectural Comparison

Aspect	BIOS	UEFI
Execution mode	16-bit real mode with BIOS interrupts	32-bit or 64-bit with flat memory access
Boot target	Reads and executes MBR then VBR	Runs EFI programs from ESP
Partition table	MBR. Up to 4 primary partitions. Extended partition for more	GPT. Many partitions by design. CRC-protected with a backup header
Max disk size	About 2 TiB with 512-byte sectors	Far beyond 2 TiB. Practical limit depends on OS and hardware
Features	Simple and universal but limited	Secure Boot, network boot, native drivers, NVMe support, etc.
Compatibility	Legacy systems	Modern default. Some firmware provides CSM for legacy support

MBR and GPT

MBR uses 32-bit addressing in the partition table. This limits disk size to about 2 TiB and allows up to four primary partitions. Logical partitions are possible through an extended partition. GPT stores 64-bit LBAs, supports many partitions by design, keeps a backup header at the end of the disk, and includes CRC checks so metadata corruption is detectable.

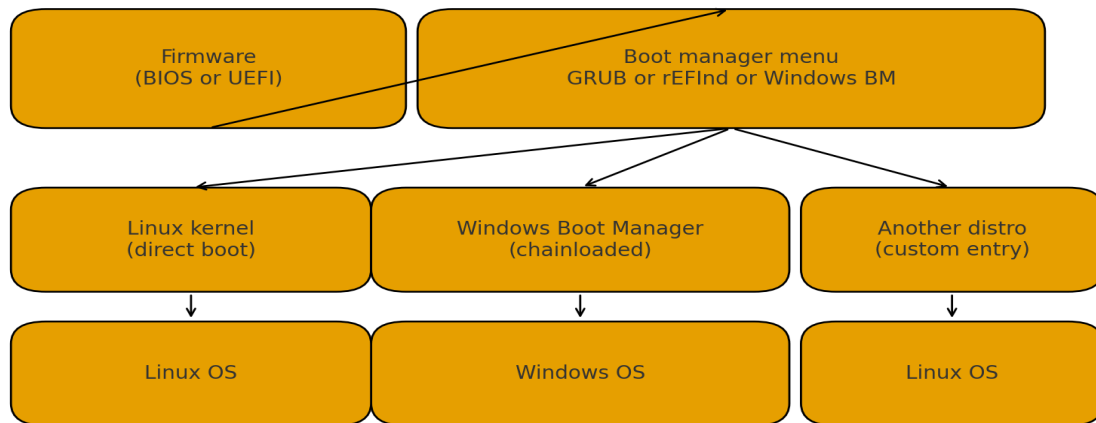
UEFI boots from an EFI System Partition. It is usually FAT32. Windows often also creates a small Microsoft Reserved Partition for GPT management. On BIOS systems that use GPT, a BIOS Boot partition is needed by GRUB to store extra code.

3. Multi-boot Systems

3.1 How multi-boot loaders manage many OS

A multi-boot loader presents a menu. It can either load a kernel directly or chainload another loader. Examples: GRUB 2, rEFInd, systemd-boot, Windows Boot Manager.

Multi-boot menu and chainloading



Menu styles and configuration

- GRUB: reads grub.cfg. Can auto-detect OS with os-prober. Supports text and basic graphics. Can boot Linux directly or chainload Windows.
- rEFInd: graphical. Scans the EFI System Partition for .efi loaders. Shows icons for each discovered entry.
- systemd-boot: simple text menu. Reads configuration from the loader entries directory in the ESP.
- Windows Boot Manager: uses the BCD store. Third-party tools or bcdedit manage entries. Often chainloaded from GRUB in mixed setups.

Common challenges

- Mixed boot mode: one OS installed in BIOS mode and another in UEFI mode. Firmware cannot switch modes during handoff.
- Secure Boot blocks unsigned loaders or modules.
- Fast startup and hibernation on Windows lock NTFS volumes and break chainloading or shared data access.
- Encrypted setups: LUKS on Linux or BitLocker on Windows require early unlock support.
- Filesystem visibility: loaders must read ext4, NTFS, or Btrfs to find kernels or chainloaders.
- Partitioning conflicts: MBR limits, primary vs logical partitions, or missing ESP.

Typical solutions

- Install every OS in the same boot mode: prefer UEFI on modern hardware. Disable CSM if possible.

- Keep one shared EFI System Partition. Place each loader in its own vendor directory. Use firmware boot order or a primary manager like rEFInd or GRUB.
- Use signed shims and signed GRUB to satisfy Secure Boot. Keep keys and db updates current.
- Turn off Windows fast startup. Fully shut down Windows before booting another OS.
- For encryption: use GRUB with cryptodisk modules or an initramfs that unlocks the root device.
- When BIOS booting with GPT: create a tiny BIOS Boot partition for GRUB. With MBR: plan primary and extended partitions early.
- Use chainloading for Windows entries. Use direct kernel boot for Linux entries to avoid extra hops.

Summary

BIOS loads a tiny MBR program that jumps to a larger loader. UEFI runs EFI programs from a dedicated partition and supports Secure Boot. MBR has strict limits. GPT removes them and improves reliability. Multi-boot works best when all systems use the same mode, a single ESP holds all loaders, and the main manager either boots kernels directly or chainloads cleanly.